

Relatório Thallium

Projeto: Sistema de Banco Digital

1. Visão Geral do Projeto

Este projeto desenvolve uma aplicação de banco digital usando Python e o framework Flask para criar uma interface web e uma API RESTful. A aplicação interage com um banco de dados MySQL para realizar operações financeiras como cadastro de clientes, criação de contas, gerenciamento de cartões, transferências, pagamentos de boletos, depósitos, saques e solicitação de empréstimos. A arquitetura do sistema foca na segurança e na experiência do usuário, com controle de acesso por sessões e criptografia de senha.

2. Estrutura do Banco de Dados e Modelagem

O banco de dados foi modelado para suportar várias entidades que representam diferentes aspectos do sistema bancário. As principais tabelas incluem:

- **Clientes:** Armazena informações dos clientes, como `id_cliente`, `nome`, `cpf`, `email`, e `senha`. A senha é armazenada como um hash para garantir a segurança.

```
15 ● ○ CREATE TABLE Clientes (  
16     id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
17     nome VARCHAR(100) NOT NULL,  
18     email VARCHAR(100),  
19     cpf VARCHAR(11) NOT NULL UNIQUE,  
20     senha VARCHAR(255) NOT NULL  
21 );  
22
```

- **Contas:** Cada cliente possui uma ou mais contas. A tabela de contas contém o `id_cliente` como chave estrangeira, `numero_conta`, `saldo`, `data_abertura` e o tipo da conta (como corrente ou poupança).

```
● ○ CREATE TABLE Contas (  
    numero_conta INT AUTO_INCREMENT PRIMARY KEY,  
    id_cliente INT,  
    saldo DECIMAL(15, 2) DEFAULT 0.00,  
    data_abertura DATE NOT NULL,  
    tipo_conta ENUM('Corrente', 'Poupança') NOT NULL,  
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente) ON DELETE CASCADE  
);
```

- **Cartões:** Armazena informações de cartões associados a cada cliente, como `numero_cartao`, `validade`, `cvv`, e `id_cliente` para referência cruzada com a tabela de clientes

```
CREATE TABLE cartoes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    numero VARCHAR(16) NOT NULL UNIQUE,
    validade VARCHAR(5) NOT NULL,
    cvv VARCHAR(3) NOT NULL,
    id_cliente INT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
);
```

- **Transações:** Mantém registros das operações realizadas por um cliente, incluindo o tipo de transação (depósito, transferência, saque) e o valor.

```
CREATE TABLE Transacoes (
    id_transacao INT AUTO_INCREMENT PRIMARY KEY,
    numero_conta INT,
    tipo_transacao ENUM('Depósito', 'Saque', 'Transferência,Pix Enviar') NOT NULL,
    valor DECIMAL(15, 2) NOT NULL,
    data_transacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    descricao VARCHAR(255),
    FOREIGN KEY (numero_conta) REFERENCES Contas(numero_conta) ON DELETE CASCADE
);
```

- **Histórico_Transações:** Registra todas as transações para permitir consultas do histórico de operações financeiras.

```
CREATE TABLE Historico_Transacoes (
    id_historico INT AUTO_INCREMENT PRIMARY KEY,
    numero_conta INT,
    tipo_transacao VARCHAR(255),
    valor DECIMAL(15, 2) NOT NULL,
    data_transacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    descricao VARCHAR(255),
    FOREIGN KEY (numero_conta) REFERENCES Contas(numero_conta) ON DELETE CASCADE
);
```

- **Empréstimos:** Registra os pedidos de empréstimo, associando-os a uma conta específica com detalhes como `valor_emprestimo`, `juros`, `prazo`, `data_emprestimo`, `data_vencimento` e status do empréstimo.

```
CREATE TABLE Empréstimos (
    id_emprestimo INT AUTO_INCREMENT PRIMARY KEY,
    numero_conta INT,
    valor_emprestimo DECIMAL(15, 2) NOT NULL,
    juros DECIMAL(5, 2) NOT NULL,
    prazo INT NOT NULL, -- prazo em meses
    data_emprestimo DATE NOT NULL,
    data_vencimento DATE NOT NULL, -- Adicionando a data de vencimento
    status ENUM('pendente', 'aprovado', 'negado') DEFAULT 'pendente', -- Status do empréstimo
    FOREIGN KEY (numero_conta) REFERENCES Contas(numero_conta) ON DELETE CASCADE
);
```

A modelagem foi desenhada para garantir a integridade referencial entre clientes, contas e transações, utilizando chaves estrangeiras e relacionamentos bem definidos.

3. Uso de Triggers e Stored Procedures

Triggers

- **Trigger de Atualização de Saldo:** O trigger de atualização de saldo pode ser configurado para ser acionado a cada operação financeira. Por exemplo, quando uma transação é inserida na tabela `Transações`, o saldo da conta é atualizado automaticamente com base no valor da transação e no tipo (crédito ou débito).
- **Trigger para Atualização do Histórico de Transações:** Cada operação financeira realizada pela conta de um cliente (como transferência, saque ou depósito) dispara um trigger que insere o registro correspondente na tabela `Histórico_Transações`. Este design garante um histórico completo e consistente.

Stored Procedures

As stored procedures facilitam a execução de operações repetitivas e complexas diretamente no banco de dados:

- **SP_Realizar_Transferencia:** Uma stored procedure que encapsula a lógica da transferência entre contas. Ela verifica o saldo da conta remetente, deduz o valor e o adiciona à conta destinatária, tudo dentro de uma transação única. Essa procedure simplifica o código da aplicação, concentrando a lógica no banco de dados e facilitando a manutenção.
- **SP_Registrar_Empréstimo:** Registra um novo empréstimo e insere os dados relacionados na tabela de empréstimos. Calcula os juros e data de vencimento de acordo com os parâmetros recebidos e mantém o status como `pendente` até a aprovação.

4. Uso de Transações e Nível de Isolamento

Para garantir a integridade das operações financeiras e evitar problemas de concorrência, foram implementadas transações que envolvem múltiplas operações SQL. As transações são utilizadas nas seguintes funcionalidades:

- **Transferências entre Contas:** A transferência é realizada em uma transação para garantir que tanto o débito da conta remetente quanto o crédito da conta destinatária ocorram de forma atômica. Em caso de falha, a transação é revertida (rollback), garantindo que nenhuma das contas seja afetada parcialmente.
- **Pagamentos de Boletos e Saques:** Os pagamentos e saques também são realizados dentro de uma transação para garantir que, uma vez que o valor seja retirado da conta do cliente, a operação é finalizada no sistema ou revertida caso ocorra uma falha.

Nível de Isolamento

O nível de isolamento escolhido para as transações é o nível **Serializable**. Este nível foi selecionado para evitar anomalias como leitura suja, leitura não repetida e leitura fantasma, especialmente importante em um sistema financeiro onde a consistência dos dados e a prevenção de operações concorrentes conflitantes são essenciais.

Com o nível Serializable, cada transação é isolada como se estivesse ocorrendo de forma independente, garantindo que os saldos e as operações financeiras não sejam influenciados por outras transações concorrentes.

5. Justificativas para Decisões de Design

1. **Triggers e Stored Procedures:** As triggers e stored procedures foram escolhidas para automatizar a lógica de negócio no banco de dados, reduzindo a quantidade de código na aplicação e aumentando a segurança e eficiência no acesso aos dados. O uso de triggers para atualizar automaticamente o saldo e o histórico de transações é uma escolha que aumenta a confiabilidade e facilita auditorias e logs do sistema.
2. **Nível de Isolamento Serializable:** O uso do nível Serializable é justificado pela natureza das operações bancárias, onde qualquer anomalia pode causar discrepâncias financeiras sérias. Esse nível de isolamento previne problemas comuns de concorrência, proporcionando uma base segura para que as transações sejam realizadas sem interferências externas.
3. **Hash de Senha e Segurança:** Para proteger as credenciais do cliente, a biblioteca `werkzeug.security` é usada para gerar e verificar hashes de senha, garantindo que as senhas não sejam armazenadas em texto simples no banco de dados. O uso de uma `secret_key` para gerenciar sessões também adiciona uma camada de segurança contra sessões mal-intencionadas.
4. **Arquitetura Modular:** A divisão das operações em funções e o uso de transações SQL permitem que o código seja organizado, de fácil manutenção e teste, o que é crucial para sistemas bancários que exigem altos níveis de confiabilidade e segurança.

6. Conclusão

Este projeto demonstra uma implementação robusta de um sistema bancário digital, com forte ênfase em segurança e consistência de dados. A utilização de triggers, stored procedures e transações SQL em um nível de isolamento alto garantem a integridade das operações, minimizando riscos de concorrência.